Paper IX

Sixth Lean Verification Block

Companion jumps and Fibonacci packet recurrences

Komorebi P. Release 0.6.0

Scope and result

This release completes Theorem 9.1 and Corollary 9.2 of the unchanged 28-page status-corrected manuscript. The preceding 28 proof modules and five supplements remain unchanged.

Twenty-two of 24 named manuscript-proved items now have formal coverage; two remain pending. Three classical inputs and two conjectures are tracked separately. The cumulative corpus has **382 named theorem declarations and 84 explicit definitions across 31 modules**.

New module	Scope	Theorems	Defs.
`CompanionPackets`	General jump and recurrence	17	2
`FibonacciPackets`	Actual Fibonacci specialization	18	4
`RecurrenceExamples`	Examples and boundary cases	14	1
Total added	Two named items	49	7

These counts include supporting lemmas and examples, not 49 independent mathematical discoveries. The principal statements are universal over their explicit inputs, not finite numerical observations.

The main mathematical connection

A sampled state $u_k = v_{Nk+j}$ evolves by the actual jump matrix C^N. Its recurrence is derived from the characteristic polynomial of that matrix. No scalar recurrence or spectral decomposition is supplied as a substitute for the desired conclusion.

For Fibonacci numbers, the state is independently defined from Mathlib's actual sequence. The Lucas sequence is independently defined by its initial values and recurrence. The trace and determinant of C^N are then proved to supply the sampled recurrence coefficients.

Important qualification

A d-dimensional state system supplies a recurrence of **order at most** d. Its minimal recurrence need not have order d. The original statement is read as the characteristic-polynomial recurrence, not a minimal-order claim. The formal equations also handle singular matrices, zero-dimensional spaces and zero step.

1. Actual matrix orbits and arithmetic jumps

Theorem 9.1; manuscript page 22

Let R be a commutative ring, I a finite coordinate type and $d = |I|$. For $C \in M_I(R)$ and $v_0 \in R^I$, define

$$\mathrm{orbit}(C, v_0, n) = C^n v_0, \qquad u_k = \mathrm{packet}(C, v_0, N, j, k) = C^{Nk+j} v_0.$$

The implementation uses actual matrix powers and matrix–vector multiplication. A separately supplied sequence v with

$$v_{n+1} = C v_n$$

is proved by induction to equal this power orbit. Thus the source-sequence formulation is connected to the construction.

The jump formula

Associativity and the laws for powers of one matrix give

$$\boxed{u_k = C^j (C^N)^k v_0 = (C^N)^k (C^j v_0).}$$

Only powers of the same matrix are interchanged. The argument does not assume that arbitrary matrices commute. In particular,

$$u_{k+1} = C^N u_k, \qquad u_{k+a} = (C^N)^a u_k.$$

The step matrix is C^N, not C. A checked example with the Fibonacci matrix and $N = 2$ rejects replacing the jump with a single original step.

Refinement of an arithmetic address

Taking another subsequence with step M and offset s yields

$$u_{Mk+s} = v_{(NM)k+(Ns+j)}.$$

The formal proof establishes this identity for the actual packet function by checking the index arithmetic. It is not inferred from a diagram or a naming convention.

Endpoints and interpretation

The algebraic identities allow $N = 0$, in which case $u_k = v_j$ is constant. Positive-modulus residue packets are the special case $N > 0$ and $0 \le j < N$. The formula also permits offsets beyond that standard range. No inverse matrix, diagonalization, eigenvalue, or characteristic-zero assumption is needed.

Principal targets.

`Companion.orbit_of_recurrence`

`Companion.companion_jump, Companion.source_companion_jump`

`Companion.packet_step, Companion.packet_advance`

`Companion.packet_refinement, Companion.packet_zero_step`

2. From the characteristic polynomial to a recurrence

Theorem 9.1, continued

Put $D = C^N$ and let

$$\chi_D(X) = \det(XI - D) = X^d + \sum_{a=0}^{d-1} c_a X^a$$

be its actual characteristic polynomial. The implementation reuses the pinned Mathlib Cayley–Hamilton theorem, expands polynomial evaluation into a finite sum and proves

$$\sum_{a=0}^{d} \chi_D[a] D^a = 0, \qquad \chi_D[d] = 1.$$

Here $\chi_D[a]$ denotes the coefficient at index a. The identities hold over any commutative coefficient ring; the zero-ring case is handled explicitly by the proof's subsingleton branch.

The complete vector recurrence

Apply the matrix identity to the actual state u_k. The already proved advance identity identifies each $D^a u_k$ with u_{k+a}. Consequently,

$$\sum_{a=0}^{d} \chi_D[a] u_{k+a} = 0, \qquad \boxed{u_{k+d} = -\sum_{a=0}^{d-1} c_a u_{k+a}.}$$

This is the desired recurrence, with its monic leading coefficient and all earlier coefficients supplied by χ_D. It is not an assumption on the packet sequence.

Every coordinate and every R-linear scalar observation $\lambda : R^I \to R$ satisfies the corresponding equation:

$$\lambda(u_{k+d}) = -\sum_{a=0}^{d-1} c_a \lambda(u_{k+a}).$$

A separate source-sequence theorem transports the recurrence back to the original v_{Nk+j} notation.

Order versus minimal order

The characteristic polynomial gives a recurrence of order at most d. For example, $C = I_2$ gives a degree-two characteristic polynomial, but every orbit is constant and obeys $u_{k+1} = u_k$. Both statements are checked. No minimal-polynomial classification is claimed.

For $d = 0$, the unique empty vector is zero and the formula remains valid. For singular matrices, Cayley–Hamilton still applies: invertibility is irrelevant. The nilpotent example $C = \left(\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right)$ has $C^2 = 0$ and $\chi_C = X^2$.

Principal targets.

`Companion.charpoly_sum_powers, Companion.charpoly_coeff_dimension`

`Companion.packet_charpoly_annihilation`

`Companion.packet_charpoly_recurrence`

`Companion.packet_coordinate_recurrence`

`Companion.packet_observable_recurrence`

3. Fibonacci and Lucas coefficients are derived

Corollary 9.2; manuscript page 22

Define the actual integer matrix and state

$$C = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}, \qquad v_n = \begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix}, \qquad v_0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

The values F_n are Mathlib's `Nat.fib`, cast into $\mathbb{Z}$. Its basic recurrence proves $v_{n+1} = Cv_n$. The second coordinate of the packet state is therefore proved to be the intended F_{Nk+j}, not a newly assumed sequence.

An independent Lucas sequence

Define

$$L_0 = 2, \quad L_1 = 1, \quad L_{n+2} = L_n + L_{n+1}.$$

Direct finite matrix arithmetic proves $C^2 = C + I$. Multiplication by C^n gives $C^{n+2} = C^n + C^{n+1}$. Taking traces shows that $\operatorname{tr}(C^n)$ satisfies the same recurrence and initial values as L_n. Induction yields

$$\boxed{\operatorname{tr}(C^N) = L_N.}$$

The implementation separately proves

$$\det C = -1, \qquad \det(C^N) = (-1)^N, \qquad L_N = 2F_{N+1} - F_N.$$

No Binet roots or diagonalizability argument is needed.

The sampled Fibonacci recurrence

The actual two-by-two characteristic-polynomial formula gives

$$\chi_{C^N}(X) = X^2 - L_N X + (-1)^N.$$

Insert these coefficients into the general packet recurrence and take the second coordinate:

$$\boxed{F_{N(k+2)+j} = L_N F_{N(k+1)+j} - (-1)^N F_{Nk+j}.}$$

The full two-coordinate state satisfies the same equation. All subtraction is in $\mathbb{Z}$, not truncated natural-number subtraction. The result holds for all natural N, j, k, including zero step and zero initial values.

Principal targets.

`FibonacciPackets.state_is_orbit, FibonacciPackets.packet_is_fibonacci`

`FibonacciPackets.trace_power_is_lucas, FibonacciPackets.det_power`

`FibonacciPackets.jump_charpoly`

`FibonacciPackets.fibonacci_packet_recurrence`

4. Exact examples and meaningful boundary checks

Example 9.3 and supporting results

The independently defined Lucas recurrence gives

$$(L_0, L_1, L_2, L_3, L_4, L_5) = (2, 1, 3, 4, 7, 11).$$

For $N = 3$, the general theorem therefore specializes to

$$u_{k+2} = 4u_{k+1} + u_k.$$

The implementation proves this for every offset j and every k, not only the displayed sample. At $j = 2$, direct evaluation of the actual Fibonacci sequence gives

$$(F_2, F_5, F_8, F_{11}, F_{14}, F_{17}) = (1, 5, 21, 89, 377, 1597).$$

This is the exact manuscript example.

The parity sign cannot be removed

For $N = 2$, the recurrence is

$$u_{k+2} = 3u_{k+1} - u_k.$$

At $j = 1, k = 0$, the correct equation is $5 = 3 \cdot 2 - 1$. Replacing the minus sign with a plus would assert $5 = 7$. The source proves the inequality, and a separate invalid-proof control attempts and rejects the false equation.

At $N = 1$ the ordinary Fibonacci recurrence is recovered. At $N = 0$, $L_0 = 2$ and the packet is constant, so the same formula becomes $u = 2u - u$.

Uniqueness from the first two packet values

For a fixed N, j, any integer sequence satisfying the proved sampled recurrence and starting at F_j, F_{N+j} agrees with F_{Nk+j} for all k. This is proved by two-step induction. It is a consequence of the established coefficients and recurrence, not a premise used to identify Fibonacci values initially.

Independent finite mirrors

A pure-Python exact-arithmetic implementation computes characteristic polynomials by the permutation determinant and states by repeated matrix application. It checks 280 matrix systems over integers, rationals and several modular rings, including composite moduli and the zero ring. Separately it computes sampled Fibonacci sequences recursively and compares them with direct Fibonacci evaluation. These are useful regression checks, not Lean extraction or universal proofs.

Principal targets.

`RecurrenceExamples.manuscript_sequence`

`RecurrenceExamples.manuscript_step_three, RecurrenceExamples.step_two`

`FibonacciPackets.fibonacci_packet_unique`

`RecurrenceExamples.even_step_wrong_sign`

`RecurrenceExamples.wrong_jump_matrix`

5. Verification, provenance and remaining work

Cumulative checks and exact scope

Lean. The fresh cumulative run uses Lean 4.34.0-rc2 and Mathlib revision

`aeb72f1e9981b6d35b426b16dca7759be3e7ba06`.

All 31 modules compile with warnings treated as errors. The inventory contains 382 theorem declarations and 84 explicit definitions; all 466 expected axiom reports must match. Allowed dependencies are `propext`, `Classical.choice` and `Quot.sound`, or subsets. The source-policy check excludes admitted proofs, added axioms and selected trust bypasses. All nine package revision pins are checked.

Controls and tests. Twenty invalid-proof controls are expected to fail for mathematical reasons, not missing imports or timeouts. The three new controls test the Fibonacci sign, the jump matrix and the trace coefficient. All 131 infrastructure and release tests pass. These include 28 new sixth-block tests and preserved earlier arithmetic tests.

Independent exact finite check	Cases
Matrix systems across seven coefficient domains	280
Jump identities and characteristic recurrences	1,680 each
One-step packet transitions	1,400
Nested packet addresses	280
Fibonacci packet sequences	357
Fibonacci scalar recurrence steps	11,067
Lucas trace, determinant and Fibonacci identities	81
Invalid numerical inputs rejected	10

Development failures. Four Lean development checks failed before correction. The errors concerned a matrix-associativity step, recursive simplification, coefficient simplification and an ineffective coordinate for a counterexample. Their source snapshots and compiler logs are retained. An initial unit-test run failed two stale baseline-count assertions; those expected counts were updated, not the mathematical tests weakened.

Trust boundary. Pinned Mathlib supplies Cayley–Hamilton, characteristic-polynomial facts, matrix algebra and the basic Fibonacci recurrence. Its compiled cache is reused, not independently rebuilt. Python mirrors do not execute extracted Lean definitions. The historical numerical suite and earlier papers are not rerun. The original manuscript and all 28 upstream proof files remain unchanged.

Remaining pair. Theorem 7.5, the global odd-prime Euler finite-difference estimate, and Corollary 8.5, the uniform standard-window period, remain pending. The two conjectures are still unproved. This block does not use those claims as assumptions.

Detailed compiler logs, inventories, amendments, the completed item crosswalk, and fresh-extraction integrity evidence accompany the release. Manuscript-to-Lean correspondence is a mathematical review, not an automatically certified semantic equivalence.